

Projekt i plan wdrożenia hybrydowej gry ekonomiczno-strategicznej inspirowanej Slipways i Grand Ages: Medieval

Streszczenie wykonawcze

Najmocniejsza interpretacja tego konceptu to **graph-first mercantile city builder**: gracz nie jest monarchą ani dowódcą wojny, tylko **Kompanią** rozwijającą sieć miast, faktorii, magazynów i tras handlowych na proceduralnie generowanej mapie. Ze Slipways warto przejąć czytelny rdzeń optymalizacyjny oparty o łączenie produkcji z potrzebami, wzrost węzłów po poprawnym zbilansowaniu przepływów, tempo badań oraz scenariuszowe ograniczenie czasu; z Grand Ages: Medieval warto przejąć zakładanie miast, wieloszczeblowe drogi i porty, lokalne rynki, towary podstawowe i przetworzone, automatyczne trasy handlowe, podatki/opłaty oraz presję konkurentów. Taki miks tworzy grę o silnym „one more turn / one more quarter” bez kosztu produkcyjnego pełnego RTS-a wojennego.

1

Najbardziej pragmatyczna decyzja technologiczna dla passion projectu o dużym udziale systemów i umiarkowanych ambicjach wizualnych to **Unity 6 + C# + czysty rdzeń symulacji w osobnym assembly**, z użyciem DOTS/Burst tylko tam, gdzie naprawdę boli CPU. Unity ma dziś oficjalne filary, które pasują do tego projektu: ECS/DOTS i Job System do skalowania obliczeń, Addressables do ładowania zawartości asynchronicznie, Cloud Save do synchronizacji postępu i ewentualnych trybów asynchronicznych, wsparcie dla buildów z linii poleceń oraz Test Framework do testów integracyjnych. Dodatkowo model opłat wrócił do subskrypcji seat-based, bo Unity anulowało Runtime Fee; to zmniejsza przewidywalne ryzyko biznesowe względem okresu niepewności z lat 2023–2024. Alternatywą o bardzo dobrym stosunku koszt/elastyczność jest Godot 4.x .NET, szczególnie jeśli kluczowe są PC-first, MIT i moddowanie.

2

Największe ryzyka tej gry nie są graficzne, tylko **systemowe**: rozrost zakresu, niestabilność ekonomii, nieczytelny UX, zbyt szczegółowa symulacja agentowa oraz wejście zbyt blisko w cudzą ekspresję IP. Literatura o łańcuchach dostaw ostrzega, że nawet małe wahania popytu mogą być wzmacniane przez lead time i zamówienia w górę łańcucha; oficjalne narzędzia backendowe do triggerów działają zwykle z semantyką at-least-once i bez gwarancji kolejności; a organizacje zajmujące się IP podkreślają, że mechanika gry i idea nie są tym samym co chroniona ekspresja audiowizualna, nazwy, znaki towarowe i elementy pochodne. Dlatego raport rekomenduje: **ograniczony zakres MVP, model miast oparty o kohorty i dystrykty zamiast pełnego agent-per-person, deterministyczny seed + save delta, oraz wyraźne odcięcie się od nazw, ikonografii i UI źródeł inspiracji.**

3

Założenia produktu

Przyjmuję następujące założenia, bo nie zostały doprecyzowane: budżet indie; **PC-first** jako platforma docelowa na start; **single-player** jako rdzeń z opcjonalnym trybem asynchronicznym po MVP; styl wizualny **2.5D izometryczny** lub stylizowane low-poly/painterly; brak taktycznej walki jako głównego systemu; zespół małego studia albo jednego senior developera z ograniczonym wsparciem artystycznym, designerskim i QA.

Teza projektowa powinna brzmieć jasno: **to ma być gra o budowaniu wartości Kompanii poprzez kontrolę przepływów, nie poprzez malowanie każdego domu ani prowadzenie wojen**. Jeśli ta teza nie jest chroniona w produkcie, projekt łatwo rozpadnie się na trzy osobne gry naraz: city builder, grand strategy i ekonomiczny Excel. Najlepszy kształt dla tego konceptu to zatem: mapa jako **hybryda grafu logistycznego i lekkiej warstwy terenowej**, miasta jako **ośrodki popytu i podaży z lokalnym rozwojem**, a wynik kampanii mierzony przez **zysk, kapitalizację, kontrakty i udział w rynku**.

Poniżej priorytety funkcji dla pierwszych faz produkcji.

Priorytet	Zakres	Dlaczego teraz	Co można obciąć bez zdrady rdzenia
P0	seed mapy, zakładanie miast/faktorii, drogi i porty, 16–20 dóbr, lokalne rynki, 3 poziomy rozwoju miasta, podstawowe badania, 1 AI konkurent, save/load, 4 kluczowe overlay UI	To jest test hipotezy „czy sieć handlowa daje satysfakcję i czy gracz rozumie ekonomię”	dekoracje high-end, rozbudowana dyplomacja, online, pełny mod kit
P1	magazyny, zamówienia automatyczne, kontrakty regionalne, wydarzenia, 2–3 archetypy Kompanii, 3 AI konkurentów, kampania scenariuszowa	Zwiększa głębię, regrywalność i presję strategiczną	zaawansowane event chainy, ranked async
P2	edytor scenariuszy, modding data-first, wyzwania asynchroniczne, replaye, bardziej złożone prawo/taryfy, kosmetyki i soundtrack/DLC	Zwiększa longevity i komercyjne „drugie życie” gry	wszystko poza tym może wejść po premierze

Projekt rozgrywki i UX

Slipways samo opisuje swój rdzeń jako budowę imperium handlowego z izolowanych planet w około 60 minut; oficjalny quickstart podkreśla, że ekonomia „runs on trade”, planety tracą wydajność i dochód przy niezaspokojonych potrzebach, a po osiągnięciu progów importu/eksportu awansują do wyższych poziomów dobrobytu. Ten sam materiał wiąże badania z laboratoriami pracującymi na konkretnych zasobach, ogranicza rozgrywkę do 25 lat i opiera scoring na jakości kolonii, populacji, technologiach i szczęściu. To są świetne właściwości dla gry, która ma być analityczna, szybka i rygorystycznie czytelna.

4

Grand Ages: Medieval celuje z kolei w dużą skalę, handel, ekspansję i „economical domination”. Manual gry opisuje miasta z biurem i rynkiem jako centralnym miejscem zarządzania towarami i cenami, wybór do pięciu operacji produkcyjnych na miasto, zależność produkcji od lokalnych surowców, manualne lub automatyczne trasy kupieckie, małą pojemność kupców wzmacnianą wozami, natychmiast zakładane i stopniowo ulepszane drogi, użycie portów do skracania tras oraz podatki zależne od dobrobytu miasta. Manual wskazuje też konkurentów, neutralnych burmistrzów, magazynowanie centralne przez depoty i zależność jednostek od towarów i rynku miejskiego. To jest bogaty wzorzec dla warstwy „company ops”.

5

Proponowana mechanika hybrydowa powinna zachować **czytelność Slipways**, ale osadzić ją w **średniowieczno-handlowej strukturze miast**. Rdzeń rozgrywki proponuję ułożyć tak:

1. **Rozpoznaj teren i załóż pierwszy charter town** w miejscu z dobrym potencjałem żywności, drewna i jednego „resource differentiatora”.
2. **Specjalizuj miasto** w jednym z kilku profili: rolniczy, wydobywczy, rzemieślniczy, portowy, finansowy.
3. **Połącz miasta i zasoby** drogami, rzekami, trasami morskimi i magazynami pośrednimi.
4. **Stabilizuj popyt i podaż**, obserwując nie tylko ceny, ale też lead time, zapasy i przepustowość.
5. **Badaj technologie i rozwijaj charter perks**, które odblokowują lepsze receptury, szybsze wozy, większe składy, tańszy kredyt, wyższą wydajność pracy.
6. **Rywalizuj z NPC kompaniami** o kontrakty, lokalizacje, marże i reputację.
7. **Wygrywaj poprzez wynik firmy**, nie przez map painting: roczny zysk, kapitalizację, udział w rynku, kontrakty królewskie, poziom zadłużenia i stabilność dostaw.

Najważniejsza decyzja projektowa brzmi: **symulacja ciągła w prezentacji, dyskretna w logice**. To znaczy, że gra może wyglądać jak płynący świat z wozami i statkami, ale faktyczne systemy ekonomiczne powinny chodzić w stałym ticku dziennym lub tygodniowym, z aktywną pauzą i możliwością szybkiego przeskoku kwartału. To daje jednocześnie „życie świata” oraz deterministyczność, testowalność i łatwiejsze balansowanie.

Miasto nie powinno być pełnym, parcelowym city builderem w stylu setek budynków. Dla tego projektu bezpieczniejszy jest model **dystryktowo-słotowy**: każde miasto ma ograniczoną liczbę slotów lub kwartałów, odblokowywanych przez populację, dobrobyt i infrastrukturę. W obrębie miasta gracz buduje np. **agricultural quarter, artisan quarter, market square, dockyard, guildhall, warehouse district, counting house, university/abbey**. To daje poczucie city-buildingu, ale nie zabija tempa i scope’u.

Warunki zwycięstwa i porażki powinny być scenariuszowe, a nie tylko sandboxowe. Rekomenduję trzy klasy zwycięstw: **finansowe** (kapitalizacja lub zysk netto), **logistyczne** (utrzymanie poziomu zaopatrzenia regionu przez czas) i **hegemoniczne** (udział w rynku lub kontrakty strategiczne). Porażka powinna następować przez **bankructwo, złamanie covenantów zadłużenia, utratę charteru, długotrwałą panikę zaopatrzeniową** albo wygraną konkurenta przed graczem. Rozwój metagry powinien być niski: odblokowywane scenariusze, startowe charter types, kosmetyka i modyfikatory kampanii — ale **bez stałych bonusów statystycznych między runami**, bo to rozmywa czystość gry strategicznej.

UX ma tu znaczenie krytyczne. Materiały z Game Developers Conference ⁶ o „glanceability” i relacji UX–game design wspierają tezę, że stan gry powinien być czytelny na pierwszy rzut oka, a interfejs musi wyjaśniać nie tylko „co się stało”, ale „dlaczego”. W praktyce oznacza to: obowiązkowy overlay niedoborów, overlay cen, overlay przepustowości tras, overlay zapasów, panel „Why am I losing money?”, panel przyczyn wzrostu/spadku miasta oraz podgląd prognozy popytu na 1–4 ticki do przodu. Bez tego nawet dobra symulacja będzie wyglądała jak chaos. ⁷

Generowanie świata i topologia mapy

Podstawowa literatura procedural content generation traktuje noise, metody constraint-based, gramatyki i metody search-based jako zestaw narzędzi do różnych warstw problemu, nie jako jedną „magiczną” technikę. Dla tej gry to ważna wskazówka: **makrogeografia, lokalizacja miast i zasobów, grywalność logistyczna** oraz **mikrowzorce wizualne** powinny być generowane innymi metodami i spinane walidacją końcową. ⁸

Najlepszy wybór architektury mapy to **hybryda tile/raster + node graph**.

Model mapy	Zalety	Wady	Wniosek
Czyste tile/hex	bardzo dobre do terenu, stref, lokalnego city-buildingu, kosztów ruchu	cięższe dla globalnej ekonomii, droższe pathfindingowo i trudniejsze do czytania jako sieć biznesowa	za ciężkie jako główna warstwa symulacji
Czysty node graph	świetny do tras, przepływów, cen, AI i czytelności ekonomicznej	słabszy dla odczucia „świata”, terenów, rzek, granic i lokalnych decyzji budowlanych	za abstrakcyjne jako jedyna reprezentacja
Hybryda	teren, biomy i koszty ruchu w rastrze; miasta, zasoby i logistyka w grafie	większa złożoność implementacyjna	najlepszy kompromis dla tej gry

Rekomendowany pipeline generacji wygląda tak:

Warstwa makro: wygeneruj pola wysokości, wilgoci, temperatury, lesistości, żyzności i mineralizacji z użyciem szybkiego noise. FastNoise Lite jest dobrym kandydatem, bo oferuje wiele typów szumu, domain warp i jawny seed; jeśli pójdziesz w Godot, klasa FastNoiseLite jest dostępna bezpośrednio w engine’u. ⁹

Warstwa regionów: z rastra budujesz regiony polityczno-ekonomiczne i biomy. Praktycznie nadaje się tu Voronoi lub hekсы. W praktyce graf Delaunay dobrze reprezentuje sąsiedztwo i potencjalne połączenia, a Voronoi dobrze reprezentuje komórki, granice i regiony wpływu; to klasyczne zestawienie dla map proceduralnych. ¹⁰

Warstwa kandydatów osadniczych: użyj blue-noise / Poisson disk do próbkowania potencjalnych lokacji miast, portów i obozów wydobywczych. Każdy kandydat oceniasz przez scoring: dostęp do wody, żyzność, sąsiedztwo dwóch lub więcej potencjalnych łańcuchów, odległość od innych ośrodków, kontakt z regionem handlowym, dostęp do wybrzeża lub wąwozu przełęczowego.

Warstwa logistyczna: na wybranych węzłach osadzasz miasta, surowce i magazyny, łączysz je grafem kandydatów przez constrained Delaunay, a następnie liczysz koszt faktycznej trasy po rastrze terenu. Topologia ekonomii działa na grafie; geometria dróg i statków bierze się z cost field.

Warstwa walidacji grywalności: po wygenerowaniu świata uruchamiasz solver lub zestaw testów, które sprawdzają, czy seed ma „solvency kernel”, czyli czy w obszarze startowym istnieje możliwość zbudowania przynajmniej jednego rentownego łańcucha podstawowego i jednego przetworzonego bez ślepego deadlocka. To jest ważniejsze od „realizmu” geograficznego.

WFC warto tu stosować **tylko lokalnie**. Oryginalna implementacja WaveFunctionCollapse dobrze nadaje się do tilemap i lokalnych wzorców, a biblioteki takie jak DeBroglie dodają nielocalne constraints. To jest świetne do generowania dzielnic portowych, układów pól, dekoracyjnych kwartałów czy ruin — ale nie jako jedyny generator całego świata ekonomicznego. ¹¹

Wydajnościowo rdzeń powinien pracować na grafie, a nie na indywidualnych kaflach. Teren służy do wyznaczania kosztów, regionów i streamingu renderu, ale **tick ekonomiczny** powinien dotyczyć głównie miast, magazynów, rynków, receptur i krawędzi logistycznych. Jeżeli kiedyś zechcesz rozwinąć mapę do dużej skali, streamuj **wizualizację** chunkami, ale nie rozbijaj wcześniej symulacji na trudne w utrzymaniu shardy. W narzędziach silnikowych odpowiednikiem tej filozofii są systemy typu Addressables po stronie assetów i World Partition po stronie otwartych światów, ale dla tej gry nie potrzebujesz ich w pełnej skali „open world”. ¹²

Dla seedów, powtarzalności i zapisów stanu kluczowe jest rozdzielenie: **seed świata, wersja generatora, hash zawartości, oddzielne strumienie RNG** dla świata, eventów i AI. Projekt PCG RNG podkreśla zalety wydajności, poprawności i seekability; to bardzo dobrze pasuje do replayów, testów golden-master i ewentualnego trybu asynchronicznego. Zapis gry powinien przechowywać **deltę względem świata wygenerowanego z seeda**, a nie pełny rzut wszystkiego. ¹³

Gospodarka, łańcuchy produkcji i AI

Grand Ages rozróżnia towary podstawowe, przetworzone i luksusowe, wiąże produkcję zarówno z pobliskimi surowcami, jak i z dalszym przetwarzaniem, a miasta organizują handel przez rynek/biuro i sieć kupców. To jest dobra baza, ale na potrzeby tego projektu polecam bardziej kontrolowalny zestaw: **12-16 surowców, 8-12 dóbr przetworzonych, 4-6 dóbr luksusowych/cywilizacyjnych**. Więcej na starcie sprawi, że balans będzie droższy niż sama implementacja. ¹⁴

Projekt zasobów powinien mieć cztery warstwy:

- **Staples:** zboże, drewno, ryby, wełna, glina, kamień.
- **Regionals:** sól, żelazo, wino, futra, len, miód.
- **Manufactured:** mąka, chleb, tkanina, narzędzia, ceramika, beczki.
- **Luxury/Civic:** odzież, ozdoby, książki, piwo/wino premium, szkło, meble.

Każdy surowiec powinien mieć:

- cenę bazową,
- profil występowania biomowego,
- ciężar/objętość przewozową,
- psucie lub nie,
- kategorię potrzeb konsumpcyjnych i przemysłowych,
- listę substytutów lub półsubstytutów.

Substytuty są ważne, bo zapobiegają „twardym deadlockom”. Przykład: miasto może akceptować ryby albo chleb jako część zaspokojenia żywności; kuźnia może korzystać z drewna opałowego z gorszą wydajnością, jeśli nie ma węgla drzewnego; odzież może akceptować wełnę lub len, ale z inną atrakcyjnością dla popytu premium.

Technologie powinny być krótkie, ale wyraziste. Zamiast ogromnego drzewa rekomenduję cztery piony:

- **Eksploatacja:** lepsze plony, głębsze kopalnie, wyższe yieldy.
- **Przetwórstwo:** nowe receptury, mniejszy waste, lepsza jakość.
- **Logistyka:** większa pojemność, tańszy transport, mosty, żegluga przybrzeżna.
- **Finanse i charter:** tańszy kredyt, nowe typy kontraktów, ulgi taryfowe, większa tolerancja długu.

Ekonomia cenowa powinna być lokalna, ale stabilizowana. Podstawowy wzór może być prosty:

$$P_{t+1} = \text{clamp}(P_{\text{base}} \cdot M_{\text{scarcity}} \cdot M_{\text{distance}} \cdot M_{\text{quality}} \cdot M_{\text{event}})$$

Nie trzeba budować pełnej teorii równowagi ogólnej; wystarczy, by lokalna cena reagowała na podaż, popyt, zapasy i koszt sprowadzenia dobra. W terminach ekonomicznych to nadal odwołuje się do intuicji równowagi podaży i popytu. ¹⁵

Dochód Kompanii powinien mieć kilka niezależnych źródeł, aby gracz nie był skazany na jeden exploit:

- **marża handlowa** na przewiezionych dobrach,
- **dywidendy z zakładów** należących do Kompanii,
- **opłaty logistyczne i magazynowe**,
- **kontrakty terminowe** z miastami lub dworem,
- **udział w cłach/taryfach** w miastach charterowych,
- **premie reputacyjne** za stabilność dostaw,
- sporadycznie **specjalne koncesje** na drogi, porty lub jarmarki.

Podatki wymagają ostrożności, bo łatwo przesuwają gracza z roli firmy do roli państwa. W źródłowym Grand Ages podatki zależą od dobrobytu i są naturalnym narzędziem władzy miejskiej. W Twojej grze lepiej zamienić „taxes” na **charter fees, import duties, market stall fees i port dues**; tylko część scenariuszy może pozwalać na bezpośredni wpływ na stawkę fiskalną w mieście, i to kosztem nastrojów oraz wzrostu. ¹⁴

Logistyka i magazyny są nie dodatkiem, lecz sercem systemu. Manual Grand Ages podkreśla znaczenie wyznaczonych tras, upgradowanych dróg, portów i centralnych magazynów; literatura o bullwhip effect pokazuje, że zmienność zamówień oraz opóźnienia prowadzą do nadmiernych stanów magazynowych, złej obsługi i błędnych decyzji produkcyjnych. Dlatego Twoja gra powinna jawnie modelować: **lead time, safety stock, reorder point, przepustowość krawędzi, koszty składowania i psucie**. Bez tych elementów ekonomia będzie „ładną tabelką”, ale nie symulacją handlu. ¹⁶

Polecam też rozdzielić **rynek lokalny** od **zapasu fizycznego w magazynie**. Rynek miejski reprezentuje to, co jest natychmiast dostępne i wpływa na cenę; magazyn to bufor zarządzany politykami Kompanii. To z kolei umożliwia sensowną automatyzację: minimalne progi zapasu, priorytety odbiorców, rezerwacja przepustowości dla dóbr krytycznych oraz polityki eksportu tylko powyżej nadwyżki.

AI miast i populacji nie powinno być mikrosymulacją każdej osoby. Narzędzia pokroju UrbanSim modelują miasta przez gospodarstwa domowe, osoby i miejsca pracy, a badania agentowe spinają rynek pracy z rynkiem mieszkaniowym. To potwierdza, że kluczowe sprzężenia zwrotne to: **populacja ↔ miejsca pracy ↔ dostępność dóbr ↔ atrakcyjność lokalizacji ↔ ceny**. Dla gry indie wystarczy jednak model kohortowy: chłopci, rzemieślnicy, kupcy i elity; do tego pula miejsc pracy per sektor oraz kilka wskaźników jakości życia. ¹⁷

Rozwój miasta może wyglądać tak:

- napływ populacji zależy od nadwyżki miejsc pracy, cen żywności, bezpieczeństwa dostaw i jakości usług,
- odpływ zależy od chronicznych niedoborów, wysokich kosztów życia, przeciążenia i niestabilności politycznej,
- produktywność pracy zależy od zdrowia/logistyki/nadwyżki zapasów,
- upgrade miasta następuje przez spełnienie warunków popytowo-infrastrukturalnych, a nie samą liczbę budynków.

NPC kompanie powinny działać jako **utility AI**, nie „uczciwi gracze 1:1”. Ich cele strategiczne to: otwarcie rentownej lokalizacji, zdominowanie kategorii towaru, przejęcie kontraktu, destabilizacja marży gracza przez dumpowanie lub wykupienie bottlenecku. Lokalne decyzje AI mogą używać zazwyczaj funkcji:

$$Score = Expected\ NPV - CapEx - TransportRisk - MarketVolatility + StrategicBonus$$

To w zupełności wystarczy, jeśli AI ma kilka osobowości: agresywny ekspander, ostrożny logistyk, monopolista luksusów, oportunistą portowy.

Technologia, architektura i dane

Poniżej porównanie opcji silnikowych.

Opcja	Mocne strony	Główne ryzyka	Trafność dla tej gry	Werdykt
Unity 6	C#, dojrzały workflow tools-first, Addressables, sensowna ścieżka do DOTS/Burst, oficjalny Cloud Save, dobre buildy CLI i testy	vendor risk, seat subscription, łatwo przesadzić z DOTS na starcie	bardzo wysoka	rekomendowane
Godot 4.x .NET	MIT, niski koszt wejścia, bardzo dobry PC-first, wbudowany FastNoiseLite, proste PCK/ZIP do modów/DLC	mniejszy ekosystem komercyjny, słabsze gotowe live services, więcej „zrób sam”	wysoka	mocny plan B
Unreal Engine 5	pełny dostęp do source, World Partition, MassEntity, świetny high-end 3D	cięższy pipeline, ergonomia 2D/2.5D mniej naturalna, royalty powyżej progu	średnia	nie rekomenduję dla v1
Silnik własny	pełna kontrola, potencjalnie maksymalna deterministyczność	ogromny koszt narzędzi, assetów, buildów, edytorów i wsparcia	niska	odrzuć

Ocena powyżej wynika z oficjalnych właściwości platform. Unity ma dziś oficjalne ECS/Entities, C# Job System/Burst, Addressables, Cloud Save, buildy z linii poleceń i Test Framework; Godot oferuje .NET, TileMapLayer, FastNoiseLite, system zapisów oraz runtime loading PCK/ZIP, a sam silnik jest na MIT; Unreal daje World Partition, MassEntity, Paper2D i pełny dostęp do source, ale dla produktu systemowego tej klasy jego największe przewagi leżą raczej w stronę cięższych produkcji 3D niż w stronę związanej ekonomii izometrycznej. Epic utrzymuje też model royalty 5% od lifetime gross revenue ponad 1 mln USD dla produktów objętych royalty-based license. ¹⁸

Moja rekomendacja wykonawcza jest konkretnie taka: **Unity 6, ale nie „full DOTS from day one”**. Rdzeń symulacji powinien żyć w osobnym, czystym module .NET/C#, bez zależności od MonoBehaviour.

Warstwa engine'owa powinna obsługiwać wejście, kamerę, rendering, UI, audio, asset loading i most do symulacji. DOTS/Burst dokładasz tylko do:

- batchingowego przeliczania produkcji,
- krótkiego pathfindingu na grafie,
- aktualizacji wielu transportów,
- benchmarków i headless soak tests.

Jeżeli jednak prawdziwym priorytetem jest **zero kosztów licencyjnych, pełna kontrola i moddowanie data-first**, Godot 4.x .NET pozostaje bardzo racjonalną alternatywą; oficjalna ścieżka ładowania PCK/ZIP w runtime jest wprost opisana jako sensowna dla DLC i bezproblemowego moddowania. ¹⁹

W zakresie sieci i platform zalecam następujący porządek:

- **MVP**: całkowicie offline, lokalny save.
- **Wersja produkcyjna**: cloud save i synchronizacja między urządzeniami.
- **Post-MVP**: wyzwania asynchroniczne, dzienne seedy, ghost companies, leaderboardzy scenariuszy.
- **Nie robić przed sukcesem MVP**: synchroniczny multiplayer lub wspólny świat ekonomiczny.

Oficjalna dokumentacja Cloud Save wskazuje, że dane mogą być współdzielone między urządzeniami i że usługa może wspierać interakcje graczy ze światem nawet wtedy, gdy nie są online jednocześnie. Jednocześnie dokumentacja triggerów zaznacza at-least-once delivery i brak gwarancji kolejności, więc każde zdarzenie backendowe musi być idempotentne, numerowane i możliwe do ponownego przetworzenia bez korupcji stanu. To jest bardzo ważne, jeśli kiedykolwiek dodasz asynchroniczne kontrakty, giełdy albo wydarzenia globalne. ²⁰

Dane projektowe powinny być **content-first i wersjonowane**. Rekomenduję taki układ:

- **authoring**: JSON/YAML albo arkusze eksportowane do JSON,
- **walidacja**: JSON Schema + generator typów,
- **runtime content**: serializowany blob/bundle,
- **save**: kompresowany JSON lub MessagePack,
- **migration**: jawne saveVersion + contentHash + worldGenVersion.

Wspieraj moddowanie najpierw na poziomie **data mods**, nie code mods. Dla Unity oznacza to zewnętrzne pliki definicji, asset bundle/addressable packs dla grafiki/audio i walidowany import. Dla Godot ścieżka PCK/ZIP jest jeszcze prostsza, ale nawet w Unity pierwsza wersja modu powinna ograniczać się do receptur, eventów, scenariuszy, opisów, ikon i prefabów bez arbitralnego wykonywania obcego kodu. Oficjalna dokumentacja Addressables i AssetBundles potwierdza, że to sensowna ścieżka do dynamicznego ładowania assetów. ²¹

Architektura wysokiego poziomu powinna wyglądać tak:

```
flowchart TB
    UI[Presentation UI]
    APP[Application Layer\nCommands Queries Undo]
    SIM[Simulation Core\nPure C# Domain]
    WG[World Generation]
    ECO[Economy Markets Pricing]
    LOG[Logistics Routing Capacity]
```



```

CITY[City Growth Workforce Demand]
AI[AI Companies Events]
PERSIST[Persistence\nLocal Save Cloud Sync]
CONTENT[Content Database\nDefs Recipes Events]
TOOLS[Editor Tools\nBalancing Scenario Authoring]
TEST[Headless Runner\nBenchmarks Regression]
TELEMETRY[Telemetry KPIs]

UI <--> APP
APP --> SIM
APP --> PERSIST
CONTENT --> WG
CONTENT --> ECO
CONTENT --> CITY
CONTENT --> AI
TOOLS --> CONTENT
SIM --> WG
SIM --> ECO
SIM --> LOG
SIM --> CITY
SIM --> AI
TEST --> SIM
SIM --> TELEMETRY
PERSIST --> UI

```

Przykładowe schematy danych dla obiektów gry:

```

{
  "resourceDef": {
    "id": "grain",
    "tier": "basic",
    "tags": ["food", "agriculture", "storable"],
    "basePrice": 10,
    "weight": 1.0,
    "spoilDays": 0,
    "biomeWeights": {
      "plains": 1.0,
      "riverlands": 0.7,
      "hills": 0.2
    },
  },
  "substitutes": ["fish"]
},
"recipeDef": {
  "id": "bread",
  "buildingType": "bakery",
  "inputs": [
    { "resourceId": "grain", "amount": 2 }
  ],
  "outputs": [
    { "resourceId": "bread", "amount": 1 }
  ]
}

```

```

    ],
    "workforce": {
      "peasants": 2,
      "artisans": 1
    },
    "baseDays": 7,
    "requiresTech": "milling",
    "byproducts": [],
    "pollution": 0
  },
  "cityDef": {
    "id": "market_town",
    "baseDistrictSlots": 4,
    "maxDistrictSlots": 10,
    "growthThresholds": {
      "village": 0,
      "borough": 150,
      "city": 500
    },
    "needs": ["food", "fuel", "cloth"],
    "prosperityModifiers": {
      "tradeDemand": 1.1,
      "migrationAttraction": 1.05
    }
  }
}

```

Przykładowy schemat save file:

```

{
  "saveVersion": 3,
  "contentHash": "sha256:9b7c0a...",
  "worldGenVersion": "0.9.0",
  "worldSeed": 128734221,
  "sessionRng": {
    "world": 123456789,
    "events": 987654321,
    "ai": 42424242
  },
  "calendar": {
    "year": 7,
    "dayOfYear": 143
  },
  "company": {
    "cash": 12450,
    "debt": 3000,
    "reputation": 62,
    "charterType": "merchant_league"
  },
  "cities": [

```

```

{
  "id": "city_001",
  "level": "borough",
  "population": {
    "peasants": 110,
    "artisans": 36,
    "merchants": 12,
    "elite": 3
  },
  "districts": ["market", "granary", "bakery", "dockyard"],
  "stock": {
    "grain": 84,
    "bread": 28,
    "wood": 53
  },
  "priceState": {
    "grain": 11.2,
    "bread": 15.4,
    "wood": 8.8
  }
},
],
"routes": [
  {
    "id": "route_017",
    "fromNode": "city_001",
    "toNode": "city_004",
    "mode": "road",
    "capacityPerDay": 12,
    "reservedFor": ["grain", "bread"]
  }
],
"events": [
  {
    "id": "evt_harvest_failure_002",
    "state": "active",
    "daysRemaining": 21
  }
],
"fogOfWar": {
  "discoveredNodes": ["city_001", "mine_004", "port_002"]
}
}

```

W save'ach krytyczne są trzy pola: **saveVersion**, **contentHash** i **worldGenVersion**. Dzięki nim możesz migrować stan gry, wykrywać niezgodność modów i odtwarzać świat z seeda zamiast przechowywać wszystko w chmurze. Dokumentacje zapisów i cloud save'ów po obu stronach — engine'owej i backendowej — dobrze wspierają taki model. ²²

Plan realizacji, operacje i biznes

Poniżej proponuję trzy makro-sprinty produkcyjne, każdy w horyzoncie około 3 miesiące. To dobrze odpowiada projektowi prowadzonemu przez senior developera z pomocą agenta Codex.

Faza	Horyzont	Deliverables dla agenta Codex
Fundament	0–3 miesiące	repo bootstrap, ADR-y, czysty rdzeń symulacji, seed generator, graf logistyczny, 8–10 zasobów, 6–8 receptur, city founding, economy tick runner, save/load v1, CLI benchmark runner, testy deterministyczne
Vertical slice / MVP	3–6 miesięcy	16–20 dóbr, 3 poziomy miast, magazyny, porty, drogi T1–T3, research v1, 1 AI konkurent, ledger finansowy, overlaye UI, scenariusz kampanii, balans seed corpus, buildy CI
Beta / pre-release	6–9 miesięcy	wydarzenia, kontrakty, 2–3 AI kompanie, polityki automatyzacji, tuning KPI, cloud save, wyzwania asynchroniczne opcjonalnie, data-driven mod support, lokalizacja, optymalizacja i telemetry dashboards

Żeby agent Codex był skuteczny, backlog powinien być pisany **modułami**, nie „feature blobami”. Dobre zadania początkowe to:

- `WorldGen.Core` : raster + region + node graph + walidacja seedów,
- `Economy.Core` : ceny, popyt, receptury, zapasy, throughput,
- `Logistics.Core` : routing, capacity, lead time, shipment tokens,
- `CitySim.Core` : populacja, miejsca pracy, upgrade miasta,
- `AI.Company` : utility scoring, plany ekspansji, polityki cenowe,
- `Persistence.Core` : serializacja, migracje, hash stanu,
- `UI.Overlays` : niedobory, ceny, throughput, przyczyny zysku/straty,
- `Tools.Balance` : import contentu, batch simulation, raporty KPI.

Zespół minimalny dla sensownego tempa wygląda tak:

Rola	Średnie FTE	Kiedy najbardziej potrzebna	Komentarz
Gameplay/Simulation Developer	1.0–1.5	od dnia 1	rdzeń projektu
Technical Designer / Economy Designer	0.5–1.0	od fazy Fundament	balans, KPI, tuning receptur
UI/UX Designer	0.3–0.7	od końca Fundament	ta rola ma krytyczny wpływ na grywalność
Generalist Artist / Environment Artist	0.5–1.0	od MVP	miasta, ikony, map tiles, props
QA	0.25 → 1.0	rosnąco od MVP	regresja ekonomii, seed corpus, usability
Sound / Composer	0.1–0.25	później	kontraktowo lub part-time

Jeśli pracujesz solo z okresowym outsourcingiem, realistyczny zakres to **12–18 miesięcy** do „naprawdę grywalnego” produktu z dobrą ekonomią. Jeśli masz 4–6 osób, można celować w **9–12 miesięcy** do mocnego MVP lub wczesnego accessu.

Wydajność i testy muszą być zaprojektowane od początku. Unity oficjalnie wspiera buildy batchmode z linii poleceń, własne skrypty buildów i Test Framework do Edit/Play mode; `dotnet test` obsługuje czysty moduł symulacji poza silnikiem, a GitHub Actions daje standardowy CI/CD z cache’owaniem zależności i artefaktów. Dla tego projektu oznacza to praktycznie: każdy commit uruchamia unit testy ekonomii, testy właściwości dla generatora i serializerów, benchmark jednego korpusu seedów, a nightly pipeline generuje 500–1000 map i raportuje KPI typu time-to-profit, frequency-of-bankruptcy, unmet-demand-ratio, AI-win-rate i median route profit. ²³

Najważniejsze klasy testów to:

- **deterministic generation tests:** ten sam seed daje ten sam hash świata,
- **property-based tests:** brak ujemnych zapasów, brak cykli długu bez eventu, brak nieskończonego zysku z jednej pętli bez kosztu,
- **golden save/load tests:** save-load-save zachowuje hash stanu,
- **soak tests:** 200+ ticków na wielu seedach bez driftu i deadlocków,
- **playability tests:** w obszarze startowym istnieje co najmniej jedna ścieżka do dodatniego cash flow,
- **UX telemetry tests:** gracz powinien dojść do pierwszej rentownej trasy w założonym oknie czasu.

Pipeline assetów powinien wspierać szybkie wariacje proceduralne, nie fotorealizm. W praktyce najlepszy zestaw to: **Blender Geometry Nodes** do generowania wariantów budynków, dróg, ogrodzeń i propsów; **Substance 3D Painter** do tekstur i materiałów; oraz opcjonalnie **Houdini Engine** do bardziej złożonych narzędzi autorskich, zwłaszcza jeśli chcesz budować proceduralne zestawy miast, portów albo dróg już w edytorze Unity. Ważne: Houdini powinno być **narzędziem produkcyjnym**, a nie runtime dependency. ²⁴

Model monetyzacji rekomenduję jako **premium game first**. Dokumentacja platform sprzedażowych wspiera zarówno DLC, jak i mikrotransakcje, ale ten gatunek lepiej znosi: podstawową grę premium, później **scenariuszowe DLC**, soundtrack, supporter pack lub edytor/scenario pack. Nie polecam F2P ani gameplay-affecting microtransactions, bo psują czytelność ekonomii, psują zaufanie do symulacji i wypychają projekt w live-ops, którego scope jest radykalnie większy. ²⁵

Kwestie prawne trzeba potraktować serio, ale bez paniki. Materiały U.S. Copyright Office ²⁶ wprost wskazują, że **idea gry, tytuł i metody gry nie są chronione prawem autorskim**, natomiast World Intellectual Property Organization ²⁷ podkreśla, że gra wideo to złożony pakiet kodu, grafiki, muzyki, znaków, fabuły i innych warstw IP. Z praktycznego punktu widzenia oznacza to: możesz inspirować się mechaniką i strukturą problemu, ale nie kopiuj nazw, słownictwa, UI layoutu, ikonicznych wzorów towarów, SFX, identyfikacji wizualnej ani treści fabularnych. Jeśli projekt ma iść komercyjnie, przygotuj też EULA dla modów/UGC i prosty proces notice-and-takedown. ²⁸

Jeżeli wdrożysz cloud save albo funkcje asynchroniczne, potrzebujesz czytelnej polityki prywatności i minimalizacji danych osobowych. Oficjalna dokumentacja Cloud Save mówi wprost o aspektach prywatności i zgodności, a brytyjski organ ochrony danych przypomina, by jasno komunikować, kto ma dostęp do plików i na jakich warunkach dane są przechowywane. Dla tej gry najlepiej byłoby trzymać w chmurze wyłącznie identyfikator konta platformy, save blob, metadane seedów i wyniki wyzwań — bez zbędnej analizy osobowej. ²⁹

Najwyższy priorytet źródłowy dla agenta Codex mają następujące materiały. Większość z nich jest po angielsku, bo taka jest dominująca forma oficjalnych dokumentacji i źródeł pierwotnych:

1. **Oficjalny quickstart i karta sklepu Slipways** — najlepszy materiał do odtworzenia rytmu pętli handlowej, badań i progresji dobrobytu. ³⁰
2. **Manual i karta sklepu Grand Ages: Medieval** — źródło dla magistral handlowych, ról miast, towarów, kupców, dróg, portów i podatków. ³¹
3. **Procedural Content Generation in Games** oraz tekst o celach i wyzwaniach PCG — rdzeń metodologii generacji świata i walidacji grywalności. ⁸
4. **FastNoise Lite, WaveFunctionCollapse i DeBroglie** — praktyczne biblioteki/algo do noise i lokalnych constraints. ³²
5. **Dokumentacja Unity Technologies** ³³ dotycząca DOTS, Burst, Addressables, Cloud Save, buildów CLI i testów — jeśli wybierzesz Unity. ³⁴
6. **Dokumentacja Godot Engine** ³⁵ dotycząca .NET, FastNoiseLite, TileMapLayer, zapisów i PCK/ZIP — jeśli wybierzesz plan B lub chcesz inwestować w moddowanie. ³⁶
7. **Materiały z Game Developers Conference** ⁶ o proceduralności, ekonomii i UX** — szczególnie te o sustainable economies, procedural generation i glanceability. ³⁷
8. **Materiały prawne WIPO i U.S. Copyright Office** — nie do projektowania mechaniki, ale do pilnowania granic inspiracji i publikacji komercyjnej. ³⁸

Końcowa rekomendacja brzmi więc jednoznacznie: **buduj małą, ostrą hipotezę produktu** — proceduralna mapa, miasta jako popytowo-produkcyjne węzły, sieć logistyczna jako główna zabawka, firma jako podmiot ekonomiczny, nie państwo — i dopiero potem dokładaj warstwy. Jeśli ten rdzeń będzie działał, projekt ma szansę być nie tylko grywalny, ale też wyjątkowo wdzięczny dla iteracji wspomaganej przez agenta Codex.

¹ ⁴ <https://store.steampowered.com/app/1264280/Slipways/>
<https://store.steampowered.com/app/1264280/Slipways/>

² ³⁴ <https://unity.com/dots>
<https://unity.com/dots>

³ <https://www.jstor.org/stable/2634565>
<https://www.jstor.org/stable/2634565>

⁵ https://store.steampowered.com/app/310470/Grand_Ages_Medieval/
https://store.steampowered.com/app/310470/Grand_Ages_Medieval/

⁶ ³⁰ <https://slipways.net/quickstart/>
<https://slipways.net/quickstart/>

⁷ <https://gdcvault.com/play/1024925/Building-Games-That-Can-Be>
<https://gdcvault.com/play/1024925/Building-Games-That-Can-Be>

⁸ ³³ ³⁵ <https://link.springer.com/book/10.1007/978-3-319-42716-4>
<https://link.springer.com/book/10.1007/978-3-319-42716-4>

⁹ ³² <https://github.com/Auburn/FastNoiseLite>
<https://github.com/Auburn/FastNoiseLite>

¹⁰ https://www.researchgate.net/publication/228587129_Delaunay_Triangulation_Algorithm_and_Application_to_Terrain_Generation
https://www.researchgate.net/publication/228587129_Delaunay_Triangulation_Algorithm_and_Application_to_Terrain_Generation

- 11 <https://github.com/mxgmn/WaveFunctionCollapse>
<https://github.com/mxgmn/WaveFunctionCollapse>
- 12 21 <https://docs.unity3d.com/6000.4/Documentation/Manual/com.unity.addressables.html>
<https://docs.unity3d.com/6000.4/Documentation/Manual/com.unity.addressables.html>
- 13 <https://www.pcg-random.org/paper.html>
<https://www.pcg-random.org/paper.html>
- 14 16 26 31 https://download.kalypsomedia.com/manuals/Manual_GAM-EN.pdf
https://download.kalypsomedia.com/manuals/Manual_GAM-EN.pdf
- 15 <https://books.core-econ.org/the-economy-v1/book/text/leibniz-08-04-02.html>
<https://books.core-econ.org/the-economy-v1/book/text/leibniz-08-04-02.html>
- 17 <https://cloud.urbansim.com/docs/general/documentation/urbansim.html>
<https://cloud.urbansim.com/docs/general/documentation/urbansim.html>
- 18 <https://docs.unity3d.com/Packages/com.unity.entities%401.4/manual/index.html>
<https://docs.unity3d.com/Packages/com.unity.entities%401.4/manual/index.html>
- 19 https://docs.godotengine.org/en/stable/tutorials/io/runtime_file_loading_and_saving.html
https://docs.godotengine.org/en/stable/tutorials/io/runtime_file_loading_and_saving.html
- 20 <https://docs.unity.com/en-us/cloud-save>
<https://docs.unity.com/en-us/cloud-save>
- 22 https://docs.godotengine.org/en/stable/tutorials/io/saving_games.html
https://docs.godotengine.org/en/stable/tutorials/io/saving_games.html
- 23 <https://docs.unity3d.com/6000.3/Documentation/Manual/build-command-line.html>
<https://docs.unity3d.com/6000.3/Documentation/Manual/build-command-line.html>
- 24 https://docs.blender.org/manual/en/latest/modeling/geometry_nodes/index.html
https://docs.blender.org/manual/en/latest/modeling/geometry_nodes/index.html
- 25 <https://partner.steamgames.com/doc/store/application/dlc>
<https://partner.steamgames.com/doc/store/application/dlc>
- 27 28 38 <https://www.copyright.gov/register/tx-games.html>
<https://www.copyright.gov/register/tx-games.html>
- 29 <https://docs.unity.com/en-us/cloud-save/privacy-and-consent/overview>
<https://docs.unity.com/en-us/cloud-save/privacy-and-consent/overview>
- 36 https://docs.godotengine.org/en/stable/tutorials/scripting/c_sharp/index.html
https://docs.godotengine.org/en/stable/tutorials/scripting/c_sharp/index.html
- 37 <https://gdcvault.com/play/1028982/Building-Sustainable-Game-Economies-The>
<https://gdcvault.com/play/1028982/Building-Sustainable-Game-Economies-The>